

# Phase 2 Documentation – Fabulari Chat System

3813ICT Full Stack Development

Hadi Alkhub s5331680 Workshop: Wednesday 3pm

GitHub: <https://github.com/HadiAlk4/Chat-App-System>

October 2026

## 1 Overview

Fabulari is a MEAN-stack chat application: MongoDB (native Node driver), Express 5, Angular 22 (standalone components, PaperCSS) and Node.js, with Socket.io for real-time chat and live request queues. Users chat in rooms that belong to groups. Three roles, Super Admin, Group Admin (GA) and User, control what each person can see and do. Phase 1 kept most data in a JSON file or in mock arrays. In Phase 2 every screen reads and writes MongoDB through the API.

**Running it:** start MongoDB on localhost:27017; cd backend && npm start (port 3000); cd frontend && npm start (port 4200).

## 2 Specifications and Requirements

Requirements come from the brief, the client meeting (Week 2) and the published clarifications. Status reflects the code on `main`.

| Area          | Requirement                                                                                                                                                                                               | Status |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Stack         | MEAN stack. MongoDB replaces <code>fakeData.json</code> . Socket.io handles real-time events.                                                                                                             | Done   |
| Auth          | Basic email and password login. Passwords are hashed with <code>bcrypt</code> . OAuth is not used.                                                                                                        | Done   |
| Password rule | At least 8 characters, alphanumeric only, at least one uppercase letter. Checked on signup and password change.                                                                                           | Done   |
| Bootstrap     | The first account registered becomes the only Super Admin. Signup shows a note when the database has no users.                                                                                            | Done   |
| Super Admin   | Handles requests only. Cannot chat, read chat history, join or propose groups. Creates a group only by approving a user proposal, and the proposer becomes the first GA. Cannot delete or ban themselves. | Done   |
| Global ban    | Approving an account deletion removes the user and all their messages, and adds their email to a permanent ban list. Banned emails cannot log in or register again.                                       | Done   |
| Audit log     | Every Super Admin decision is stored with a timestamp. A dedicated page filters by action type and date range.                                                                                            | Done   |
| Groups        | Unique group names. Any user can view all groups and request to join. No limit on how many groups a user joins or administers.                                                                            | Done   |
| Age limit     | Min age applies to every room in the group. Users under it cannot request to join. Raising it removes members who are too young, but cannot remove the last GA.                                           | Done   |
| GA settings   | Edit name, description ( $\leq 250$ chars), min age and theme without Super Admin approval. A confirm dialog lists the changes first.                                                                     | Done   |

| Area           | Requirement                                                                                                                                                               | Status   |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Rooms          | GA creates, renames and deletes rooms. Users propose rooms and the GA approves or rejects them. Zero to unlimited rooms per group.                                        | Done     |
| Members        | GA can promote members to GA. A group always keeps at least one GA. GA sees the allowed and banned member lists.                                                          | Done     |
| Group bans     | Bans are permanent and there is no un-ban. Users can ask a GA to ban someone. A ban on a GA goes to the Super Admin instead.                                              | Done     |
| Step down      | A GA cannot step down if they are the only GA or have pending Super Admin requests.                                                                                       | Done     |
| Group deletion | A GA requests deletion and the Super Admin approves it. The group and its data are removed, and the GA is demoted if they administer no other group.                      | Done     |
| Requests       | Rejections need a reason. Pending requests cannot be cancelled. Users see pending and rejected requests in Request History.                                               | Done     |
| Messaging      | Single-channel, single-thread. Text plus PNG, JPEG and GIF images up to 2MB. No external links. Shows sender name and timestamp. Users can delete their own messages.     | Done     |
| History        | Only the last 5 messages are loaded when entering a room. Live messages are not capped while the user stays. A new room starts blank.                                     | Done     |
| Real-time      | Join and leave toasts (closable with X), live participant sidebar, GA badge in the sidebar, and a toast when a join request is accepted or denied.                        | Done     |
| Cascade delete | When an account deletion is approved, that user's messages disappear from every open chat straight away.                                                                  | Done     |
| Theme          | The group's theme (light/dark) applies to its chat rooms. A user can override it with a personal theme that applies across all groups.                                    | Done     |
| Profile        | Username (changeable), email (fixed, unique ID), password change with current password, DOB/age, role, profile picture ( $\leq 2$ MB), memberships. Profiles are private. | Done     |
| Responsive     | Desktop first and usable at tablet width (PaperCSS grid).                                                                                                                 | Done     |
| HTTPS          | Messages served over HTTPS.                                                                                                                                               | Not done |

**Known limitations.** The server runs on HTTP. The API trusts the username sent by the client and has no session token, so role checks happen in the Angular guard and in selected routes. The upload filter also accepts WebP, which the spec does not list.

## 3 Server-side API

Base URL `http://localhost:3000`. Bodies are JSON (uploads use `multipart/form-data`). Most responses have the form `{ ok, message, ...data }`. Validation failures return `ok:false` with a 400, 403 or 404 status. Server errors return 500. GET list endpoints return an array.

### 3.1 REST endpoints

| Method                          | Endpoint                            | Input                               | Behaviour                                                                                                |
|---------------------------------|-------------------------------------|-------------------------------------|----------------------------------------------------------------------------------------------------------|
| <i>Authentication and users</i> |                                     |                                     |                                                                                                          |
| POST                            | <code>/api/auth</code>              | email, password                     | bcrypt compare. Blocks banned emails. Returns the user without the password.                             |
| GET                             | <code>/api/signup/first-user</code> | —                                   | <code>{ firstUser }</code> . True when no users exist yet.                                               |
| POST                            | <code>/api/signup</code>            | username, email, password, dob, age | Checks the password rule, the ban list and duplicate emails. The first user becomes <b>super-admin</b> . |

| Method                                                                                     | Endpoint                               | Input                                                     | Behaviour                                                                                                                               |
|--------------------------------------------------------------------------------------------|----------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| PATCH                                                                                      | /api/users/username                    | email, newUsername                                        | Must be unique. Renames the user in groups, messages and requests.                                                                      |
| PATCH                                                                                      | /api/users/password                    | email, currentPassword, newPassword                       | Checks the current password, then applies the rule and stores the new hash.                                                             |
| PATCH                                                                                      | /api/users/theme                       | email, isDarkMode, usePersonalTheme                       | Saves the personal chat theme override.                                                                                                 |
| <i>Groups, rooms and members</i>                                                           |                                        |                                                           |                                                                                                                                         |
| GET                                                                                        | /api/groups                            | –                                                         | All groups.                                                                                                                             |
| GET                                                                                        | /api/groups/user/:username             | –                                                         | Groups where the user is a member or admin.                                                                                             |
| GET                                                                                        | /api/groups/:groupName                 | –                                                         | One group, 404 if missing.                                                                                                              |
| PATCH                                                                                      | /api/groups/:groupName                 | groupName, groupDescription, minAge, themeColor, username | GA only (403 otherwise). Unique name, description ≤250 chars. Removes members under the new min age. A rename updates every collection. |
| POST                                                                                       | /api/groups/:groupName/leave           | username                                                  | Leave the group. The only GA cannot leave.                                                                                              |
| POST                                                                                       | /api/groups/:groupName/rooms/direct    | roomName                                                  | GA adds a room. Names are unique within the group, ignoring case.                                                                       |
| PATCH                                                                                      | /api/groups/:groupName/rooms/rename    | oldName, newName                                          | Rename a room.                                                                                                                          |
| DELETE                                                                                     | /api/groups/:groupName/rooms/:roomName | –                                                         | Delete a room.                                                                                                                          |
| POST                                                                                       | /api/rooms                             | groupName, roomName                                       | Older direct add from Phase 1.                                                                                                          |
| PATCH                                                                                      | /api/groups/:g/members/:u/promote      | –                                                         | Add the member to <b>admins</b> .                                                                                                       |
| POST                                                                                       | /api/groups/:g/members/:u/remove       | –                                                         | Remove the member. The only GA cannot be removed.                                                                                       |
| POST                                                                                       | /api/groups/:g/members/:u/ban          | –                                                         | Permanent group ban ( <b>bannedMembers</b> ). The only GA cannot be banned.                                                             |
| POST                                                                                       | /api/groups/:g/admins/:u/step-down     | –                                                         | Blocked if they are the only GA or have pending Super Admin requests. Demotes their role if they administer no other group.             |
| <i>Request queues (each has create, list, approve, reject; reject needs <b>reason</b>)</i> |                                        |                                                           |                                                                                                                                         |
| POST, GET                                                                                  | /api/group-requests                    | groupName, description, minAge, themeColor, creator       | Group proposal to the Super Admin. The name must be free. The Super Admin cannot propose.                                               |
| PATCH                                                                                      | /api/group-requests/:id/approve reject | performedBy, reason                                       | Approve creates the group with a <b>Main Room</b> and the proposer as GA. Both are audited.                                             |
| POST, GET                                                                                  | /api/join-requests                     | groupName, username                                       | Checks age, ban and duplicate requests. The Super Admin cannot join.                                                                    |
| PATCH                                                                                      | /api/join-requests/:id/approve reject  | reason                                                    | Approve adds the user to <b>members</b> .                                                                                               |
| POST, GET                                                                                  | /api/room-requests                     | groupName, roomName, username                             | Members only. The room name must be free.                                                                                               |
| PATCH                                                                                      | /api/room-requests/:id/approve reject  | reason                                                    | Approve adds the room.                                                                                                                  |
| POST, GET                                                                                  | /api/group-ban-requests                | groupName, targetUsername, requestedBy                    | Routed to the GA, or to the Super Admin if the target is a GA. Filter with <b>destination</b> .                                         |

| Method                         | Endpoint                                          | Input                             | Behaviour                                                                                |
|--------------------------------|---------------------------------------------------|-----------------------------------|------------------------------------------------------------------------------------------|
| PATCH                          | /api/group-ban-requests/:id/approve reject        | reason                            | Approve applies the permanent group ban.                                                 |
| POST, GET                      | /api/group-deletion-requests                      | groupName, requestedBy, reason    | GA only. One pending request per group.                                                  |
| PATCH                          | /api/group-deletion-requests/:id/approve reject   | performedBy, reason               | Approve deletes the group, its messages and its requests, and demotes the GA. Audited.   |
| POST, GET                      | /api/account-deletion-requests                    | username                          | The user asks to delete their account. Not allowed for the Super Admin.                  |
| PATCH                          | /api/account-deletion-requests/:id/approve reject | performedBy, reason               | Approve deletes the user and their messages and bans the email. Audited as a global ban. |
| GET                            | /api/banned-emails                                | –                                 | Permanently banned emails, newest first.                                                 |
| <i>Chat, uploads and audit</i> |                                                   |                                   |                                                                                          |
| GET                            | /api/messages/:groupName/:roomName                | query: username                   | Last 5 messages, oldest first. Returns 403 for the Super Admin.                          |
| DELETE                         | /api/messages/:id                                 | –                                 | Delete a message.                                                                        |
| POST                           | /api/upload/avatar                                | form: image, username             | Image $\leq$ 2 MB. Sets <b>profilePictureUrl</b> .                                       |
| POST                           | /api/upload/chat                                  | form: image                       | Image $\leq$ 2 MB. Returns <b>fileUrl</b> (served from /uploads).                        |
| GET                            | /api/audit-logs                                   | query: action, startDate, endDate | Filtered audit log, newest first.                                                        |

## 3.2 Socket.io events

Room channels are named `groupName:roomName`. The server ignores chat events from unknown users and from the Super Admin, and drops messages that contain `http(s)://` or `www..`

| Direction     | Event                              | Payload and effect                                                                                                                                                                                                                                                                              |
|---------------|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| client→server | join-room, leave-room              | {groupName, roomName, username}. Joins or leaves the channel and updates who is present.                                                                                                                                                                                                        |
| client→server | send-message                       | {groupName, roomName, senderUserName, content, imageUrl}. Saves to <code>messages</code> , then broadcasts it.                                                                                                                                                                                  |
| client→server | delete-message                     | {groupName, roomName, messageId}. Deletes from MongoDB, then broadcasts.                                                                                                                                                                                                                        |
| server→room   | new-message, message-deleted       | The saved message, or {messageId}.                                                                                                                                                                                                                                                              |
| server→room   | user-joined, user-left, room-users | Trigger toasts and the live participant list.                                                                                                                                                                                                                                                   |
| server→all    | <queue>-created, <queue>-resolved  | For group, join, room, group-ban, group-deletion and account-deletion requests. Queues and Request History refresh live. <code>join-request-resolved</code> triggers the accept/deny toast. <code>account-deletion-request-resolved</code> removes the deleted user's messages from open chats. |

## 4 Angular Architecture

Standalone components with routing in `app.routes.ts`. The logged-in user is kept in `sessionStorage`, so each browser tab has its own login. `authGuard` protects every route except login and signup. It checks `expectedRole` for Super Admin pages and sends a Super Admin away from `chat`, `my-memberships` and `group-settings`.

### 4.1 Components

| Component           | Route                  | Responsibility                                                                                                                                                                                                                                                   |
|---------------------|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Login               | /login                 | Posts credentials to <code>/api/auth</code> , stores the user, and routes by role (Super Admin dashboard or dashboard). Shows API errors, including the banned message.                                                                                          |
| Signup              | /signup                | Registration with age from DOB and an optional profile doodle. Shows a note when the next account will be the Super Admin.                                                                                                                                       |
| Dashboard           | /dashboard             | Lists and searches groups, proposes a group (modal), and requests to join. Groups the user is already in are hidden. Both actions are hidden for the Super Admin.                                                                                                |
| MyMemberships       | /my-memberships        | The user's groups, filtered by name and role. Opens chat, settings, or leave.                                                                                                                                                                                    |
| GroupSettings       | /group-settings        | GA-only tabs. <i>General</i> : edit and save with a confirm dialog. <i>Rooms</i> : add, rename, delete. <i>Members</i> : promote, ban, banned list. <i>Requests</i> : join, room and ban queues. Also step down and request deletion. Updates live over sockets. |
| RequestHistory      | /request-history       | The user's pending and rejected requests with reasons. Tabs to propose a room and to request a ban.                                                                                                                                                              |
| Chat                | /chat                  | Room list, last-5 history plus live messages, image attachments, delete own message, toasts, participant sidebar with GA badge, and the group or personal theme.                                                                                                 |
| UserProfileSettings | /user-profile-settings | Change username, password and theme. Upload a profile picture. Request account deletion.                                                                                                                                                                         |
| SuperAdminDashboard | /super-admin-dashboard | Live queues for group proposals, GA ban requests, group deletions and account deletions, plus the banned-email list. Every rejection needs a reason.                                                                                                             |
| SuperAdminAuditLog  | /super-admin-audit-log | Audit table filtered by action type and date range.                                                                                                                                                                                                              |

## 4.2 Services

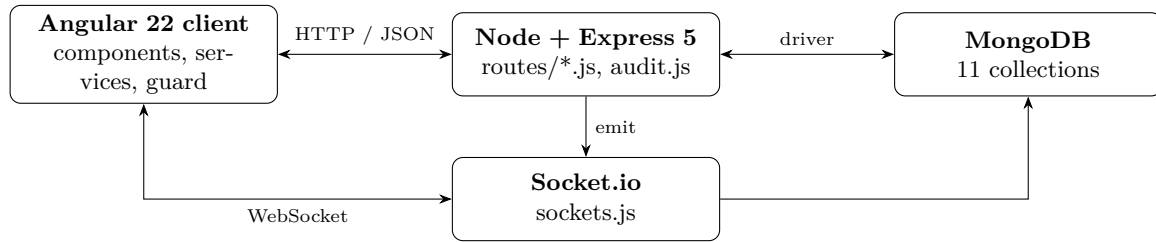
- **AuthService**: get, set and clear the session user; logout.
- **GroupService**: all group, room and member endpoints, plus every request queue (group, join, room, ban, deletion).
- **ChatService**: room history over HTTP. Emits and listens to chat socket events (`join-room`, `send-message`, `new-message`, `room-users`, and so on).
- **SocketService**: typed observables for every `*-created` and `*-resolved` request event.
- **AccountService**: account deletion requests, banned emails, and username, password and theme updates.
- **UploadService**: avatar and chat image uploads (`FormData`).
- **AuditService**: audit log query with filters.

## 4.3 Models (TypeScript interfaces)

`Group`, `GroupRequest`, `JoinRequest`, `RoomRequest`, `GroupBanRequest`, `GroupDeletionRequest`, `AccountDeletionRequest`, `BannedEmail`, `AuditLog`, `ChatMessage`. They mirror the MongoDB documents in Section 5.2. Request models share a `status` of `pending`, `approved` or `rejected`, plus `createdAt`, `reviewedAt` and `rejectionReason`.

## 5 Design Documents

### 5.1 System architecture



`app.js` builds the Express app (`createApp(io)`). `index.js` attaches the HTTP server and Socket.io. Because of this split, tests can import the app without starting a server, using a stub `io`. Business rules are pulled out into small, pure helpers in `backend/lib/` (password policy, age check, link check, last-5 window).

### 5.2 Data structures (MongoDB collections)

| Collection                           | Fields                                                                                                                                                               |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>users</code>                   | username, email (unique ID), password (bcrypt), dob, age, role ( <code>super-admin group-admin user</code> ), valid, isDarkMode, usePersonalTheme, profilePictureUrl |
| <code>groups</code>                  | groupName (unique), groupDescription, minAge, themeColor ( <code>light dark</code> ), admins[], members[], rooms[], bannedMembers[]                                  |
| <code>messages</code>                | groupName, roomName, senderUserName, content, imageUrl, timestamp                                                                                                    |
| <code>groupRequests</code>           | groupName, groupDescription, minAge, themeColor, creatorUserName, creatorEmail + request fields                                                                      |
| <code>joinRequests</code>            | groupName, username + request fields                                                                                                                                 |
| <code>roomRequests</code>            | groupName, roomName, username + request fields                                                                                                                       |
| <code>groupBanRequests</code>        | groupName, targetUsername, targetEmail, targetRole, requestedBy, requestedByRole, destination ( <code>group-admin super-admin</code> ) + request fields              |
| <code>groupDeletionRequests</code>   | groupName, requestedBy, requestedByRole, reason + request fields                                                                                                     |
| <code>accountDeletionRequests</code> | username, email, role + request fields                                                                                                                               |
| <code>bannedEmails</code>            | email, username, bannedAt, reason                                                                                                                                    |
| <code>auditLogs</code>               | timeStamp, actionPerformed, target, performedBy (insert only)                                                                                                        |

*Request fields* are status, createdAt, reviewedAt and rejectionReason. Groups store room names and usernames as arrays instead of references, so a single read loads a whole group. When a user or group is renamed, the server updates the name in every collection.

### 5.3 Storyboards

The Phase 1 storyboards (desktop and tablet) still describe the layout, so they were kept. They are in Appendix A and in `assignment_material/Fabulari Design/`. Changes made in Phase 2:

- The Group Settings *General* tab is a real form and asks for confirmation before saving.
- Request History has a *Request Ban* tab.
- The Super Admin dashboard has live GA ban and group deletion queues.
- Chat has image attachments and a delete button on your own messages.
- Every Super Admin rejection asks for a reason.

## 6 Testing

### 6.1 Tools and methodology

- **Backend unit tests**, Mocha with Node `assert` (`npm run unitTest`). Test the pure helpers in `backend/lib`.

- **Backend integration tests**, Mocha, Chai and chai-http (`npm test`). Each test calls `createApp()` against a separate `chat-app-test` database. The database is wiped before and after every test, so no test depends on another. Each endpoint has at least one success case and one failure case.
- **Angular unit tests**, Vitest with TestBed via `ng test`. Services are replaced with test doubles, so components are tested without a backend.
- **End-to-end tests**, Cypress (`npm run cypress:run`), against the real frontend and a backend started with `MONGO_DB_NAME=chat-app-test`. A `resetDb` task drops the test database before each spec.

Before any tests were written, the logic was moved into testable helpers and the app setup was split from `listen()`. The end-to-end tests found four screens that did not refresh after data loaded (login errors, dashboard, memberships, chat), and these were fixed.

**Results (run 1 Oct 2026):** backend unit 14/14 passing, backend integration 100/100 passing, Angular 27/27 passing. Cypress has 9 specs.

## 6.2 Automated tests

| Suite       | Target                                            | #  | Cases                                                                                                                                                      |
|-------------|---------------------------------------------------|----|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unit        | <code>isValidPassword</code>                      | 4  | accepts valid; rejects too short, no uppercase, symbol                                                                                                     |
| Unit        | <code>isUnderMinAge</code>                        | 3  | under min; equal to min; missing age not treated as under                                                                                                  |
| Unit        | <code>containsExternalLink</code>                 | 3  | detects http, www; allows plain text                                                                                                                       |
| Unit        | <code>latestMessagesOldestFirst</code>            | 4  | keeps latest 5; exactly 5; fewer than 5; oldest-first order                                                                                                |
| Integration | <code>/api/auth</code> , <code>/api/signup</code> | 6  | valid login; wrong password; banned login; first user is Super Admin; weak password; banned email signup                                                   |
| Integration | <code>/api/users/*</code>                         | 6  | rename; missing name; change password; wrong current password; save theme; missing theme flags                                                             |
| Integration | <code>/api/groups*</code>                         | 11 | list all / empty; user groups / none; get one / 404; leave / missing username; save settings and remove under-age members; missing description; non-GA 403 |
| Integration | rooms                                             | 8  | add / missing name; rename / missing new name; delete / 404; <code>POST /api/rooms</code> add / unknown group                                              |
| Integration | members                                           | 8  | promote / 404; remove / only GA; ban / 404; step down / only GA                                                                                            |
| Integration | group requests                                    | 8  | create / missing fields; list / empty filter; approve creates group with GA / not pending; reject with reason / no reason                                  |
| Integration | join requests                                     | 8  | create / under age; list / empty filter; approve adds member / not pending; reject with reason / no reason                                                 |
| Integration | room requests                                     | 8  | create / missing fields; list / empty filter; approve adds room / not pending; reject with reason / no reason                                              |
| Integration | group ban requests                                | 8  | create / missing fields; list by destination / empty; approve bans / 404; reject with reason / no reason                                                   |
| Integration | group deletion                                    | 8  | create / missing reason; list / empty; approve deletes and demotes / 404; reject with reason / no reason                                                   |
| Integration | account deletion, banned emails                   | 10 | create / Super Admin blocked; list / empty; approve deletes and bans / 404; reject with reason / no reason; banned list / empty                            |
| Integration | <code>/api/messages</code>                        | 5  | last 5 oldest first; missing username 400; Super Admin 403; delete / 404                                                                                   |
| Integration | <code>/api/upload/*</code>                        | 4  | avatar PNG / no file; chat PNG / no file                                                                                                                   |
| Integration | <code>/api/audit-logs</code>                      | 2  | returns logs; filters by action                                                                                                                            |
| Angular     | App                                               | 2  | creates root; renders router outlet                                                                                                                        |
| Angular     | authGuard                                         | 3  | logged out to login; user blocked from Super Admin page; Super Admin blocked from chat                                                                     |
| Angular     | Login                                             | 3  | empty fields no HTTP; user to dashboard; Super Admin to Super Admin dashboard                                                                              |
| Angular     | Signup                                            | 4  | age for past, empty and future DOB; empty form not posted                                                                                                  |
| Angular     | Dashboard                                         | 2  | proposal sent via service; joined groups hidden                                                                                                            |
| Angular     | Chat                                              | 3  | link not sent; plain text sent; history shows sender name                                                                                                  |

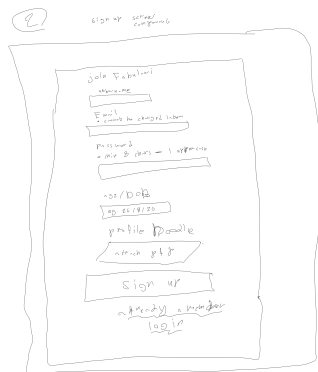
| Suite   | Target                        | # | Cases                                                                                             |
|---------|-------------------------------|---|---------------------------------------------------------------------------------------------------|
| Angular | GroupSettings                 | 4 | form filled from group; confirm before raising min age; rejection needs reason; non-GA redirected |
| Angular | RequestHistory, MyMemberships | 2 | renders a join request; renders a membership                                                      |
| Angular | SuperAdminDashboard           | 1 | renders pending proposals                                                                         |
| Angular | SuperAdminAuditLog            | 2 | renders logs; applies action and date filters                                                     |
| Angular | UserProfileSettings           | 1 | shows error on wrong current password                                                             |
| E2E     | login.cy.ts                   | 3 | logs in and fills dashboard; sends credentials to API; error on empty submit                      |
| E2E     | access.cy.ts                  | 3 | logged-out redirect; logged-in reaches dashboard; bad credentials stay on login                   |
| E2E     | admin-group.cy.ts             | 1 | Super Admin approval makes the group visible to its creator                                       |
| E2E     | chat.cy.ts                    | 2 | two users: other member's message shows their name; link message not shown                        |

## 7 Git Workflow

Every feature was built on its own branch and merged into `main` through a GitHub pull request (221 commits, PRs #1 to #52). Each PR description lists what the branch changed. Phase 2 branches, in order:

| Branch (PR)                                           | Work                                                                                           |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>feature/auth-route-guard</code> (#28)           | AuthService, route guard with roles, logout on every screen                                    |
| <code>feature/auth-and-live-groups</code> (#30, #31)  | bcrypt and Super Admin bootstrap, Group models and service, dashboard loads live groups        |
| <code>feat/group-proposal-flow</code> (#32, #33)      | Group proposal to Super Admin queue; first Socket.io events                                    |
| <code>feature/group-join-requests</code> (#34, #35)   | Join request model, routes, sockets, GA queue, Request History                                 |
| <code>refactor/backend-mongodb-structure</code> (#36) | Split backend into <code>db.js</code> and <code>routes/*</code> modules                        |
| <code>feat/room-proposal</code> (#37, #38)            | Room proposals wired to Group Settings                                                         |
| <code>feature/my-memberships</code> (#39)             | Memberships from MongoDB                                                                       |
| <code>feature/group-settings</code> (#40)             | Rooms and Members tabs on live data                                                            |
| <code>feat/chat</code> (#41, #42)                     | Messages collection, chat sockets, last-5 history, delete, live participants                   |
| <code>feature/file-attachments</code> (#43)           | Multer uploads for avatars and chat images                                                     |
| <code>account-deletion-bans</code> (#44)              | Account deletion queue, permanent email bans                                                   |
| <code>audit-log-mongodb</code> (#45)                  | Audit log collection and filtered page                                                         |
| <code>user-profile-mdb</code> (#46)                   | Username, password and theme updates                                                           |
| <code>group-admin-remaining</code> (#47)              | Unique names, settings save, age eviction, group bans, ban requests, step down, group deletion |
| <code>feat/sa-request-only-scope</code> (#48)         | Super Admin limited to requests (UI, guard, API, sockets)                                      |
| <code>feat/chat-toast-cascade-theme</code> (#49)      | Join toasts, live cascade delete, link blocking, personal theme                                |
| <code>testing</code> (#50)                            | Testable refactor, then Mocha, Chai, Vitest and Cypress suites                                 |
| <code>cleanup-bugs</code> (#51, #52)                  | GA check on settings, confirm dialog, first-user note, per-tab sessions                        |

# A Storyboards



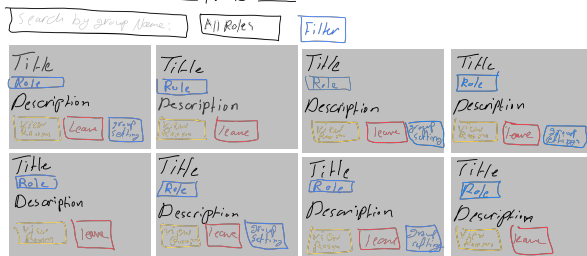
Login / Sign up



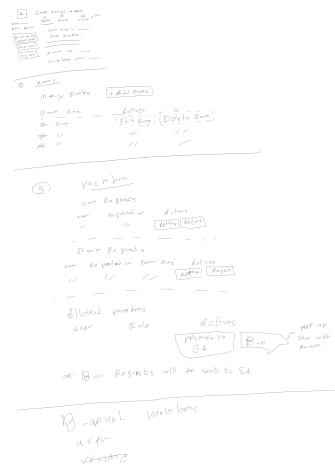
Dashboard

## My Memberships

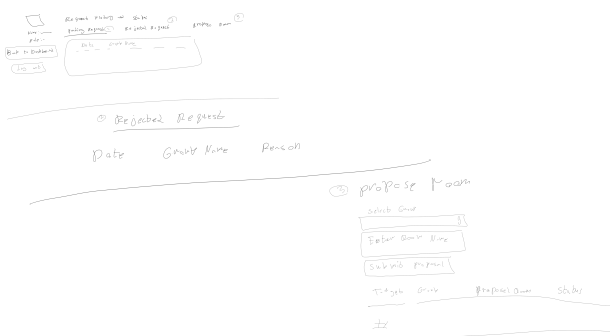
Current user: \_\_\_\_\_ | Role: \_\_\_\_\_



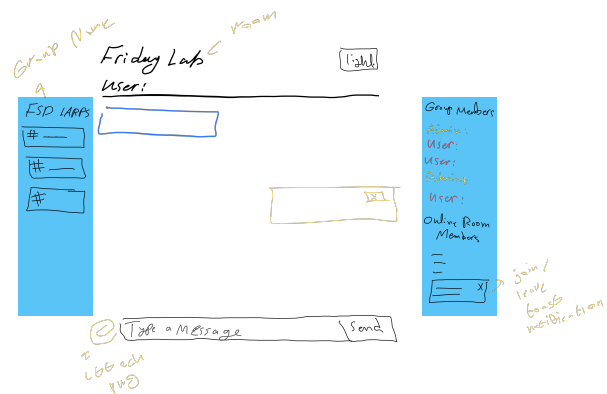
My Memberships



Group Settings



Request History



Chat

### Account Settings

User name:

#### Account Defaults:

Email Address

Birth Date

Sign-in Role

**My Membership**

#### Identity & Preferences

Name

Dark Mode Theme ☐

• It will be the dark screen.

**Profile Doodle**

would to change your / ask ?

upload a new doodle below (max 1000px)

**Upload New doodle.**

#### Security

Current password

New password

Confirm new password

**Save New Password**

Account Settings

### Super Admin Dashboard

**Group Deletion Request**

| # | Group Title | Requested by | user Role | Actions                                           |
|---|-------------|--------------|-----------|---------------------------------------------------|
| 1 |             |              |           | <input type="checkbox"/> <input type="checkbox"/> |

**Account Deletion Request**

| # | user name | Email | Role | Actions                                           |
|---|-----------|-------|------|---------------------------------------------------|
| 1 |           |       |      | <input type="checkbox"/> <input type="checkbox"/> |

**User Ban Request**

| # | user name | Email | user Role | Request |
|---|-----------|-------|-----------|---------|
|---|-----------|-------|-----------|---------|

Super Admin Dashboard

### Super Admin Audit Log

Name:  to  All Actions

Role:

**Super Admin Dashboard**

**Log out**

| #  | Timestamp | Action performed | Target ID |
|----|-----------|------------------|-----------|
| 1- |           |                  |           |
| 2- |           |                  |           |
| 3- |           |                  |           |
| :  |           |                  |           |

Super Admin Audit Log